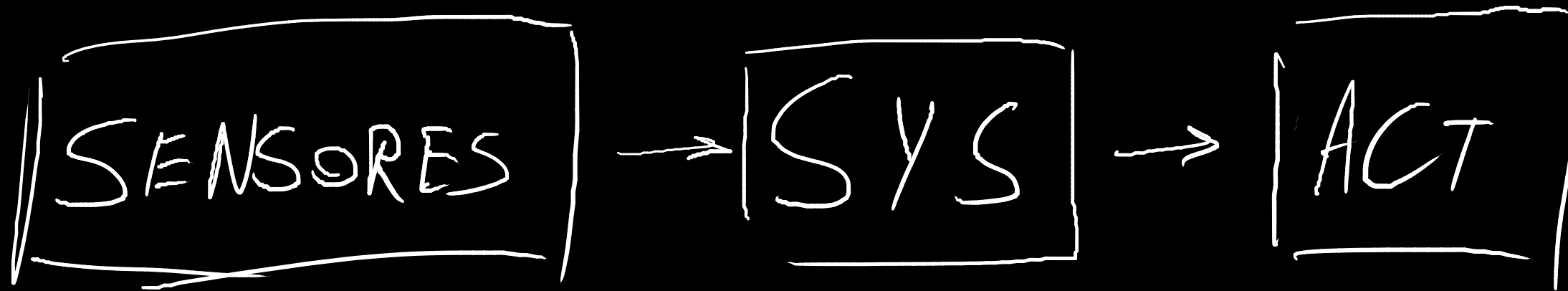
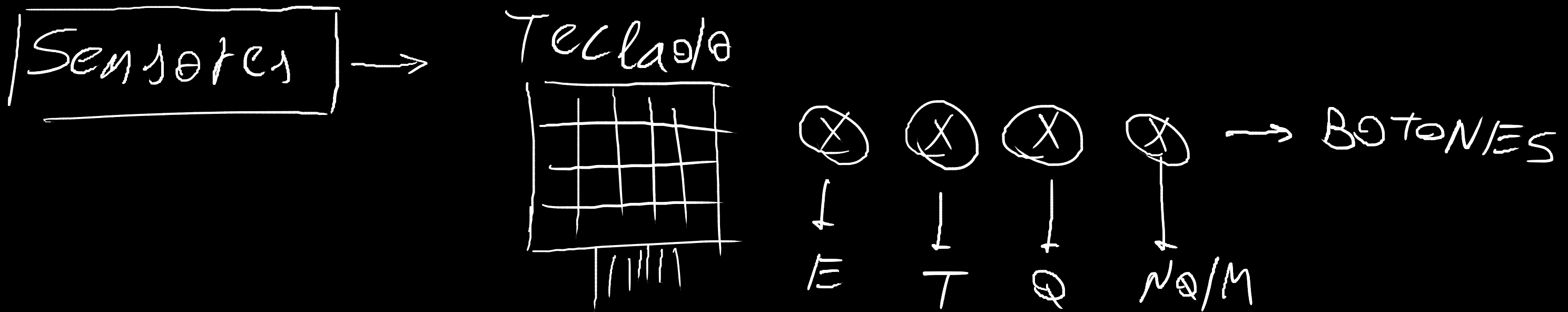


- 1) Juegan 2 jugadores. 1 Con el teclado le pasamos las cartas que se juegan. Quedan guardadas en el micro
 - 2) Cada jugador tiene que poder cantar envido
 - 3) Cada jugador tiene que poder responder (no querer o subir la apuesta)
- Agregar una LED amarilla que indique q hay que responder
- 4) Cada jugador tiene que poder irse al mazo. (si hay un envido en juego, hay q mostrar las cartas)
 - 5) Luego de cada ronda se suman los puntos, que se muestran en un LCD





1) Con el botón del truco se canta truco o se sube la apuesta (se canta retruco/vale 4)

2) El botón para no querer e irse al mazo es el mismo.

3.a) El envío es complicado:

Un jugador canta envío apretando una vez en botón E.

Un jugador canta real envío (de entrada) apretando el botón E dos veces. Para ello tiene un intervalo de, digamos, 3 segundos.

Un jugador canta falta envío (de entrada) apretando E 3 veces

El segundo jugador responde al envío. Puede responder con envío (apretando una vez), con real envío (apretando 2 veces) o con falta envío (apretando 3 veces)

El primer jugador puede a su vez responder de la misma forma.

NOTA: Todo esto se basa en el supuesto de que podemos contar cuántas veces se apretó un botón. De no ser posible, hacer:

3.b) Agregar más botones.



4) Las cartas se pasan al uC de la siguiente manera:

Nuestros supuestos: a) no se pasan cartas repetidas al uC.
b) Puede haber errores de tipeo.

->Primero se pasa el número y luego el palo.

->Los palos se pasan de la siguiente manera

Palos: A->Espada, B-> Basto, C-> Copa, D->Oro

->Los números se pasan de la siguiente manera:
Teclas del 1 al 7 -> pasan su respectivo número

Tecla 8 -> 10

Tecla 9 -> 11

Tecla 0 -> 12

Tecla # ->Enter (enviar la carta al sistema)

Tecla* ->Borrar

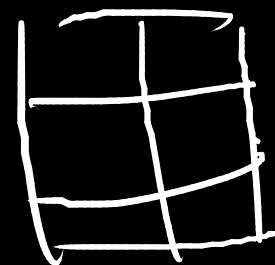
De esta forma, al sistema solo llegans Strigns con 2 chars

Cómo puede haber errores, debe haber una etapa de validación de las cartas.

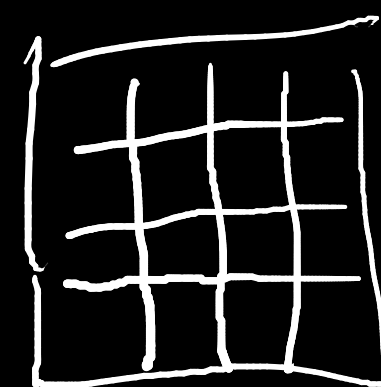
Posibles errores: dos números seguidos
dos palos seguidos
un palo y después el número.

IDEA: un bloque switch con todos los casos correctos posibles, al final del bloque switch está el caso default que maneja los errores.

MATRIZ RONDA



NUM
PAD



CADAC ARTA VA A

Sistema:

Un gran problema es comparar las cartas xq el orden es caprichoso.

Soluciones: a) ¿Tabla de búsqueda? --> Investigar

b) función comparar_carta (carta_1, carta_2); ==> Va a ser un bloque if larguísimo. no conviene

c) cuando se carga cada carta, asignarle una prioridad

propuestas: (esto no es definitivo)

existe el tipo palo_t

carta_t: {palo_t palo, int valor, ¿int prioridad?}

jugador_t_ {int puntos, carta_t carta1, carta2, carta3}

sistema_t {jugador_t jugador0, jugador1, envido, truco}

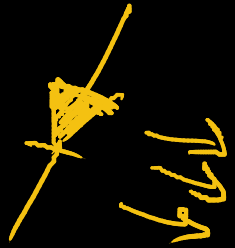
Es muy temprano para definir todo esto en detalle.

El sistema va a recibir las cartas en orden y ya validadas por la etapa anterior.

Una vez recibidas las cartas y los cantos el sistema hace 2 cuentas:

a) si se cantó envido: quién ganó el envido y cuantos puntos le suma

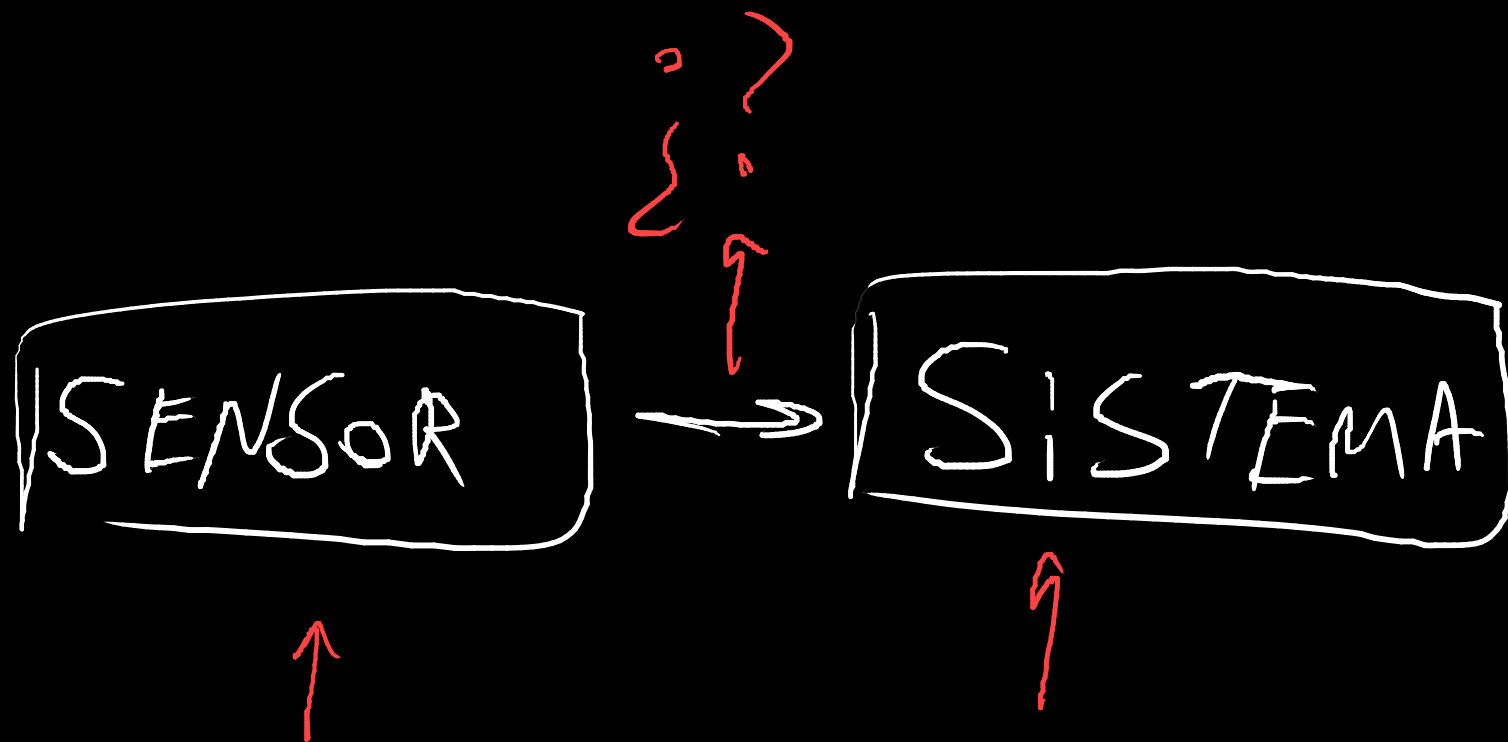
b) quién ganó la mano y cuántos puntos le suma (truco, retruco, etc)

Actuadores: LCD y LED's → 

10/7

KEYPAD-SCAN() → [0 0 0 1 ... 0 0] x16

El sensor manda las lecturas tan crudas como sea posible



11/7

Propuesta #1:

```
typedef enum {Basto, Copa, Oro, Espada} palot_t
typedef enum {NO_SE_CANTO_T (NSCT), TRUCO, RETRUCO, QV4} truco_t
typedef enum {NO_SE_CANTO_ENV(NSCE), E, EE, RE, FE} envido_t
typedef enum {CJ1, CJ2, NINGUNO} canto_t
typedef enum {TJ1, TJ2} turno_t (turno j1 o turno j2)
typedef enum {MJ1, MJ2} mano_t
```

```
typedef struct carta_t:
int valor
palo_t palo
```

```
typedef struct jugador_t
carta1
carta2
carta3
int puntos
```

jugador->carta->palo

```
typedef struct ronda:
int mano (0, 1 ó 2) //por qué mano vamos
mano_t mano = MJ1(en principio) //qué jugador es mano
turno_t turno = TJ1 (en principio) //de quién es el siguiente turno
jugador_t jugador1
jugador_t jugador2
```

Mejor propuesta para el envideo:

-> cuando un jugador canta envideo aparece un menu el display. 4 opciones:
E, EE, RE, FA.

Con el botón del envideo iteras por el menú

Con el botón de Quiero le das al enter

truco_t truco

canto_t canto_truco = ninguno (en principio) //quién cantó truco (si alguien cantó)

envido_t envido

canto_t canto_envido = ninguno(en principio) //quién cantó envido (si alguien cantó)

Propuesta #2

tenes dos vectores

cartas jugador 1:

carta1

carta2

carta3

cartas jugador 2:

carta1

carta2

carta3

struct typedef puntos

puntos j1

puntos j2

INICIAR

ST-INIT

MALX →

+carta: Validar(carta)

OK ✓

→ cargarla a J1

→ Avanzar el turno

cargamos la 2^{da} carta → Validamos (✓ ó X)

if(turno == TJ1)

ronda->jugador1->carta->palo

ronda->jugador1->carta->valor

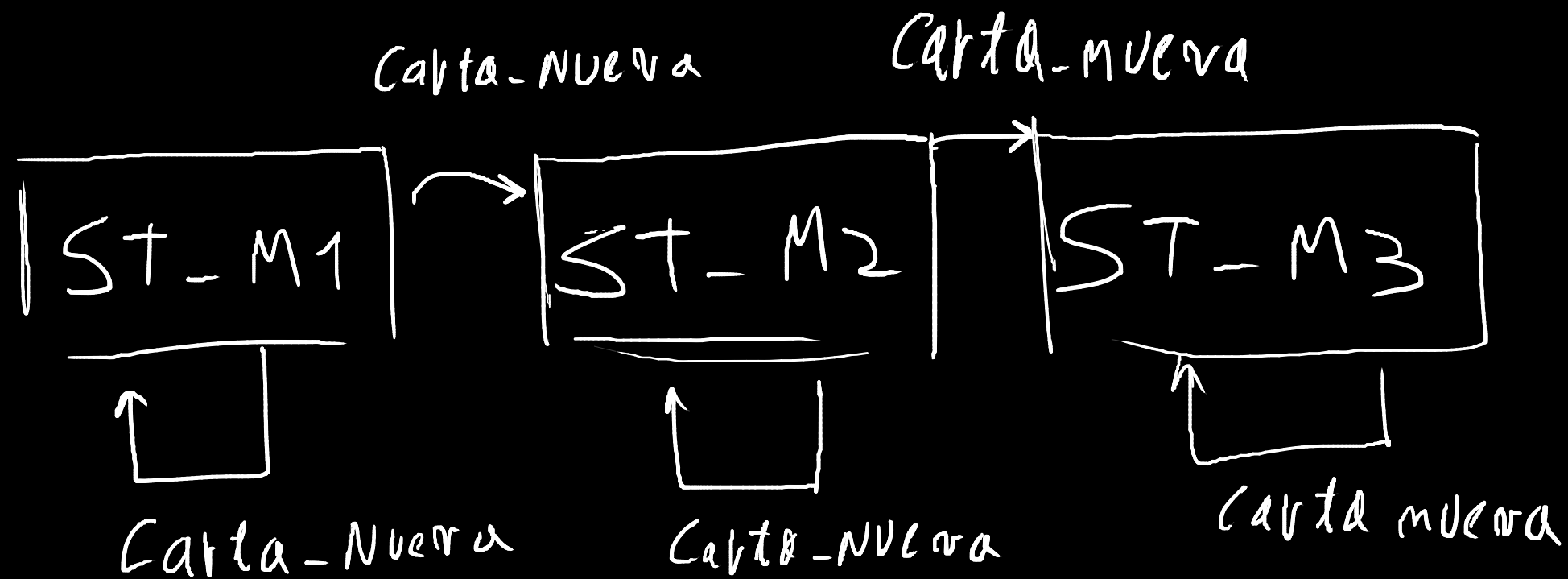
else

ronda->jugador2->carta->palo
ronda->jugador2->carta->valor

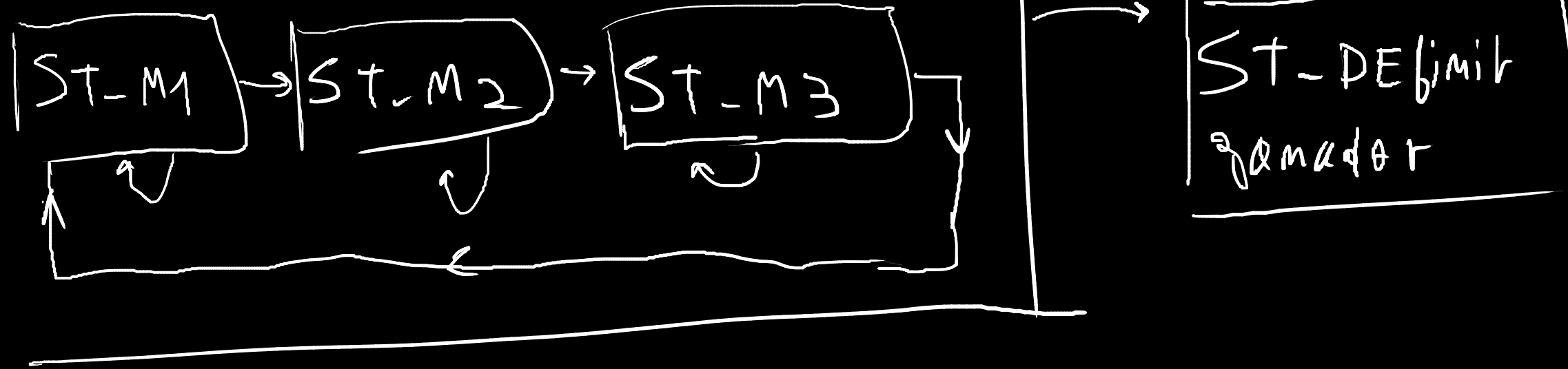
si turno fuera 0 o 1 y en ronda hubiera un
vector de jugador_t

ronda->jugadores[turno]->carta->palo
ronda->jugadores[turno]->carta->valor

Mejor: se puede usar el tipo turno_t
ronda->jugadores[ronda->turno]->carta->palo



ST- RONDA

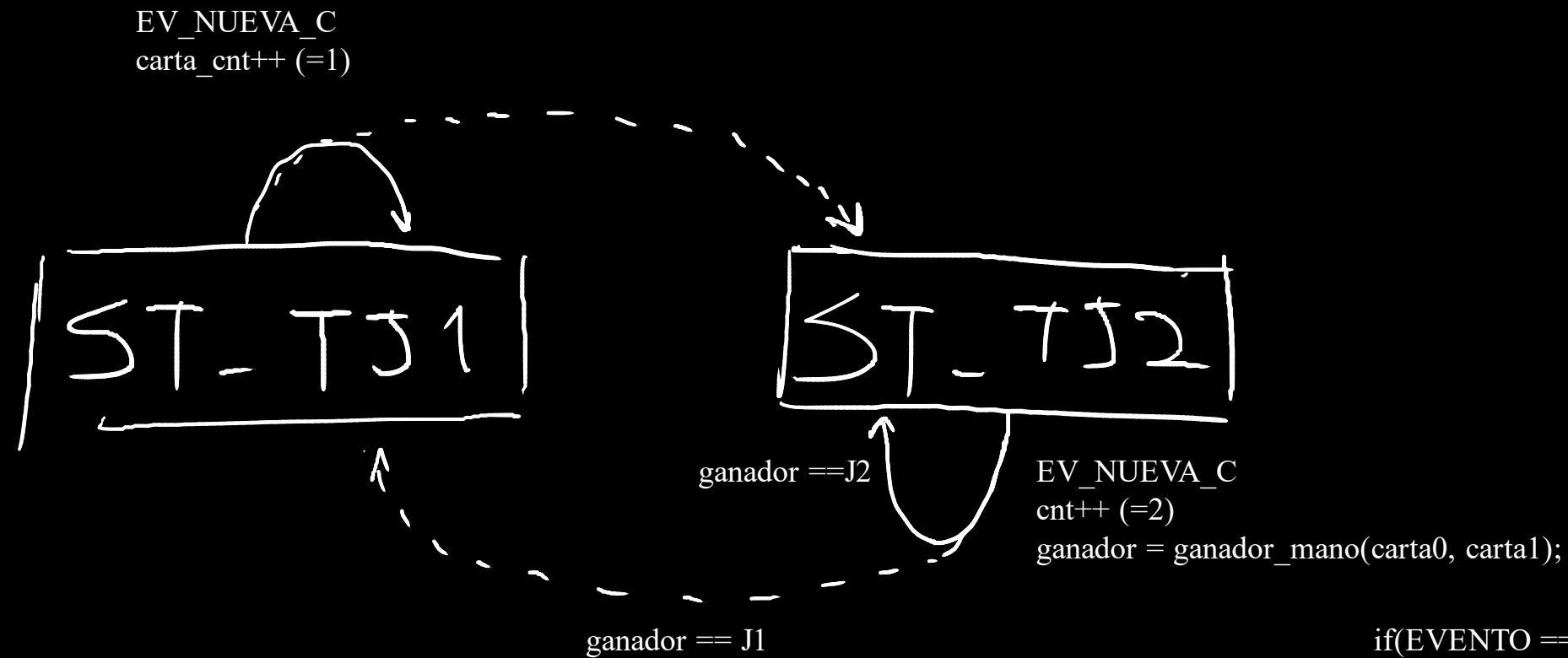


OTRA IDEA: el turno de cada jugador es un estado:

Eventos: EV_NUEVA_CARTA
EV_CARTA ERROR

Variable: int carta_cnt --> Contador de cuántas cartas se cargaron

→ Para una
carta mal
cargada



¿Que es este tipo de transiciones?

2 casos posibles. 1) Llega una nueva carta y hay que cambiar de turno
2) Llega una nueva carta y hay que llamar a ganador_mano() para ver si hay que cambiar de turno

```
evnet = EV_CARTA_NUEVA;
```

```
switch(event){
```

```
  case EV_CARTA_NUEVA:
```

```
    if(cnt_carta%2 == 0){
```

```
      ganador = ganador_mano(cartas_n, cartas_(n-1));
```

```
      if(ganador == J1) state = ST_TJ1
```

```
      else state = ST_TJ2;
```

```
    }
```

```
    cartas_cnt++
```

```
    if(state == TJ1) state = ST_TJ2
```

```
    if(state == TJ2) state = ST_TJ1
```

```
}
```

¿Cómo accedo a las cartas?

Depende de cómo definamos las estructuras

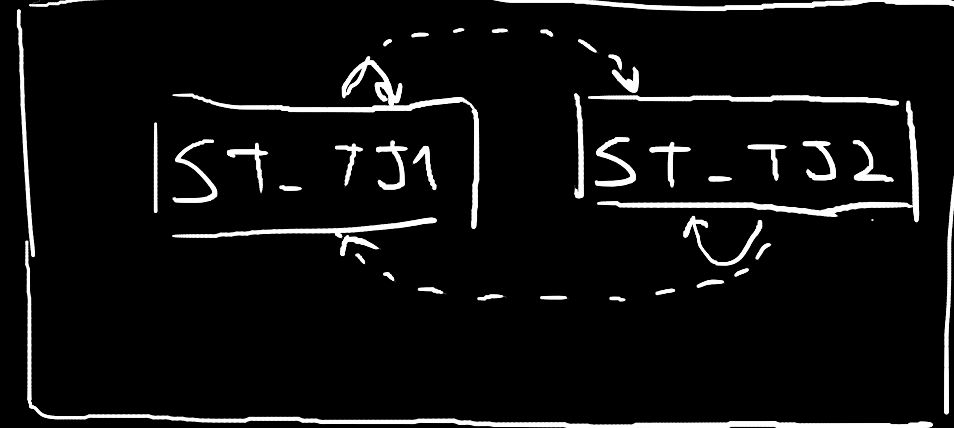
```
if(EVENTO == EV_CARTA_N && cartas_cnt multiplo de 2){
    ganador = ganador_mano(cartas0, cartas1);
    if(ganador == J1){
        STATE = ST_TJ1
    }
}
```

EV_TRUCO/RETRUCO/QV4
EV_E/EE/RE/FE

IDEA: 1) podemos guardar el último estado en una variable
2) si guardamos qué jugador es mano (que vamos a necesitar para el envío) podemos hacer una cuenta falopa con qué jugador es mano y cuántas cartas se jugaron (si son pares o impares)

Para el truco

ST-TURNOS



EV-TR
canto = TRUCO

EV-Q

ST-TR-PENDIENTE

EV-RET
canto = RET

EV-QV4
canto = QV4

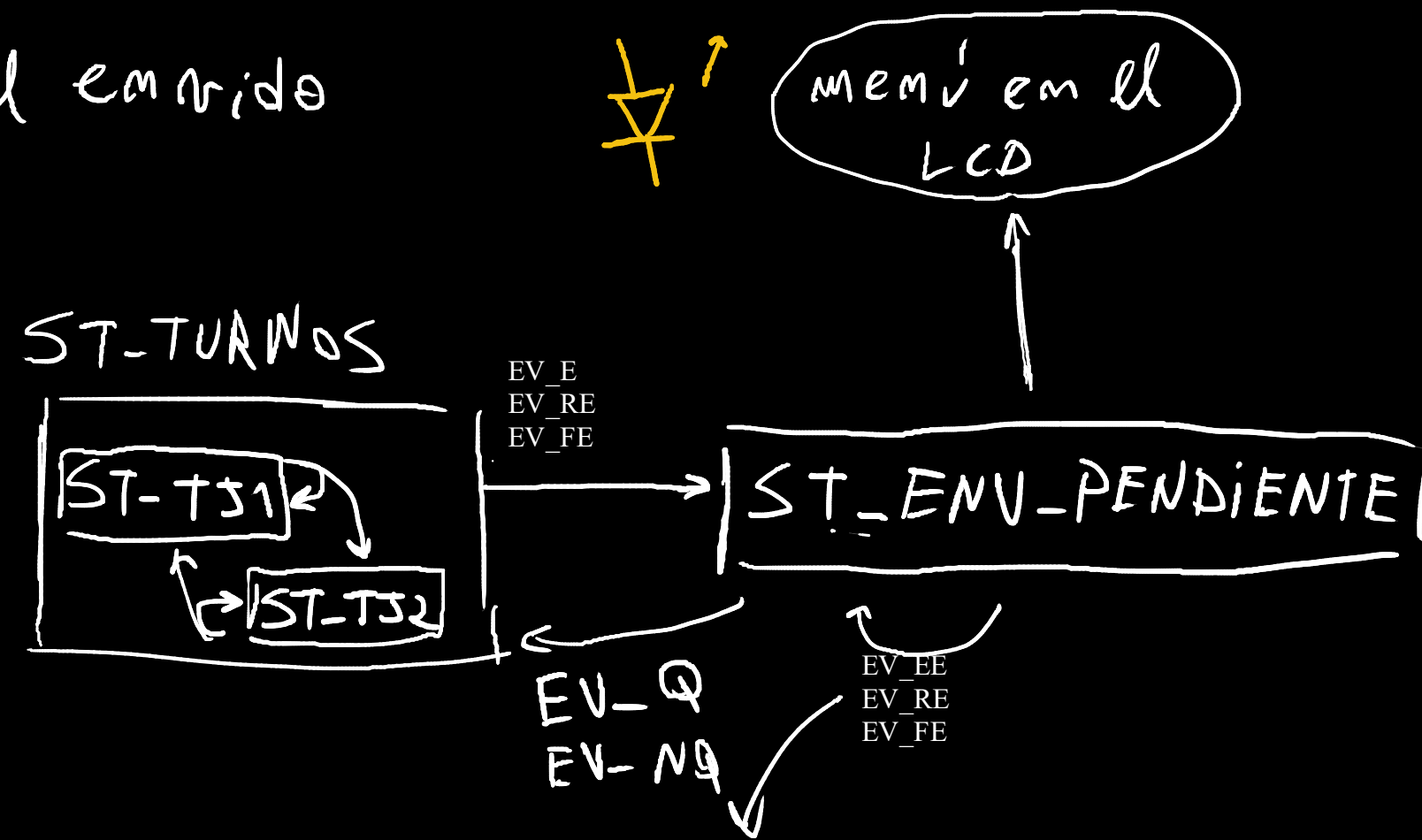
EV-NQ

ST-FIN-DE-RONDA

¿A qué estado vuelve?
Necesita memoria/shallow history



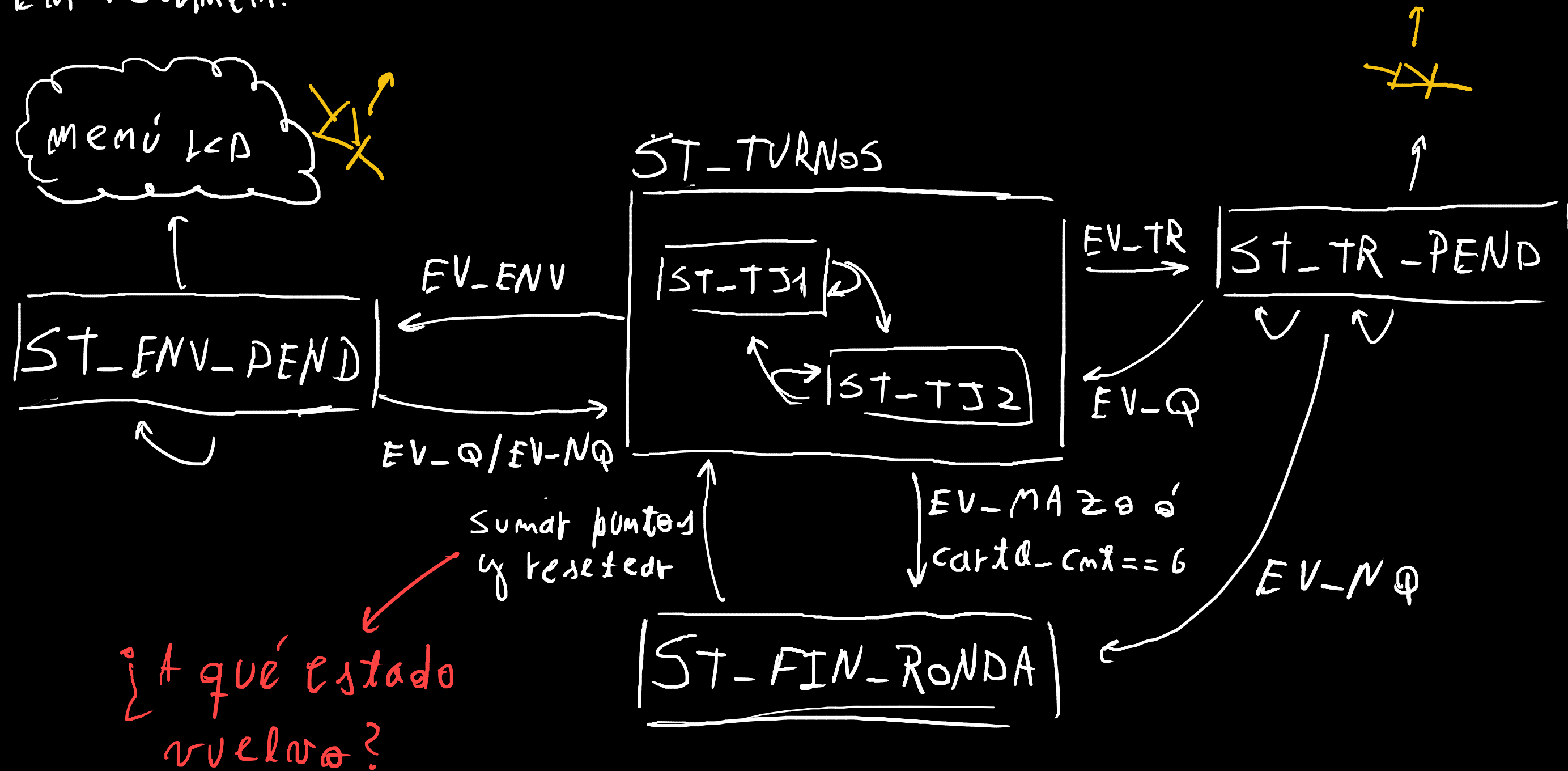
Data el emvrido



Idea: Ni lo
consideremos y
asumamos q' los jugadores
no van a bajar la apuesta

⊖ I ⊖ : No se puede
bajar la apuesta

Em resumen:



OTRA IDEA: SUB-estados por cada canto de truco o envide